FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Thu 25 February 2016 - Theory Part

LAST NAME (capital letters pls.):

FIRST NAME (capital letters pls.):

MATRICOLA: SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
  1. Theory (80%): Syntax and Semantics of Languages
     - regular expressions and finite automata
     - free grammars and pushdown automata
     - syntax analysis and parsing methodologies
     - language translation and semantic analysis
  2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the following extended regular expression R, i.e., a regexp with non-standard operators, over the three-letter alphabet $\Sigma = \{\ a,\ b,\ c\ \}$:

$$R = \big(\ a\ \mid\ b\ \big)^* \big[\ a\ c^*\ b\ \big]$$

where the square brackets "[ ]" indicate optionality.

Answer the following questions:

(a) By using the Berry-Sethi method (BS), find a deterministic finite state automaton A equivalent to expression R.

(b) Determine if automaton A is minimal or not, and if necessary minimize it.

(c) Say if expression R is ambiguous or not, and justify your answer.

(d) Say if language $L\,(R)$ is local or not, and justify your answer.

(e) (optional) In the case expression R is ambiguous, find an equivalent regular expression R' that is not ambiguous, and provide a brief and informal but convincing explanation of the non-ambiguity of R'.

---

2 Free Grammars and Pushdown Automata 20%

1. Consider the two following languages L_1 and L_2 over the three-letter alphabet $\Sigma = \{ a, b, c \}$:

$$
\begin{aligned}
L_1 &= \left\{ a^n \, c^m \, b^{2n} \mid \quad m > 0 \ \wedge \ n \geq 0 \right\} \\
L_2 &= \left\{ a^{2n} \, c^{2m} \, b^n \mid \quad m > 0 \ \wedge \ n \geq 0 \right\}
\end{aligned}
$$

   Answer the following questions:

   (a) Write two *BNF* grammars G_1 and G_2, with axioms S_1 and S_2, that generate languages L_1 and L_2, respectively.

   (b) Write an *ambiguous BNF* grammar G that generates the union language $L = L_1 \cup L_2$. Show that this grammar G is ambiguous, by exhibiting an ambiguous sentence s generated by G and all the syntax trees of s.

   (c) (optional) Design a *non-ambiguous BNF* grammar G' for the same language L, and provide an adequate justification for the absence of ambiguity. Furthermore, show the unique syntax tree of G' for the same sentence s of the previous point.

---

2. Consider the fragment of a programming language targeted at the computation of operations on one-dimensional vectors and on scalars, i.e., integers. This language features the following syntactic structures:

   - There are variables of vector (1-D) and scalar (integer) type, which are identified by uppercase and lowercase identifiers, e.g., A, B, ..., and a, b, ..., respectively.
   - A vector can also be explicitly written as a non-empty list of scalars, enclosed in graph brackets "{" and "}", and separated by commas ",", e.g., { a, b, a, c }.
   - There are arithmetic expressions, which can contain vector and scalar variables, with the addition operation "+".
   - There are subexpressions, enclosed in round brackets "(" and ")".
   - The associativity of the addition operator "+" is unspecified.
   - There are assignments with a variable on the left side, the assignment operator ":=" in between, and an expression on the right side.
   - An expression that contains a vector operand (variable name or list of scalars) may not be assigned to a scalar variable.
   - The language fragment consists of a possibly empty list of assignments, and each assignment is terminated by a semicolon ";".

   Sample language fragment:

   ```
   a := b + c;

   B := a + B + c;

   B := { a, b, c };

   A := A + { c, d } + (c + C);
   ```

   Answer the following questions:

   (a) Write an extended (*EBNF*) grammar G, not ambiguous, that models the language fragment described above.

   (b) (optional) Suppose the syntax of this language must be modified in such a way that, for instance, an expression like A + a + b + B is parsed as A + (a + b) + B, to be able to pre-compute as many purely scalar operations as possible (for they are inexpensive in comparison to the vector operations), and in this way to save valuable computing time. Here are two more such sample assignments:

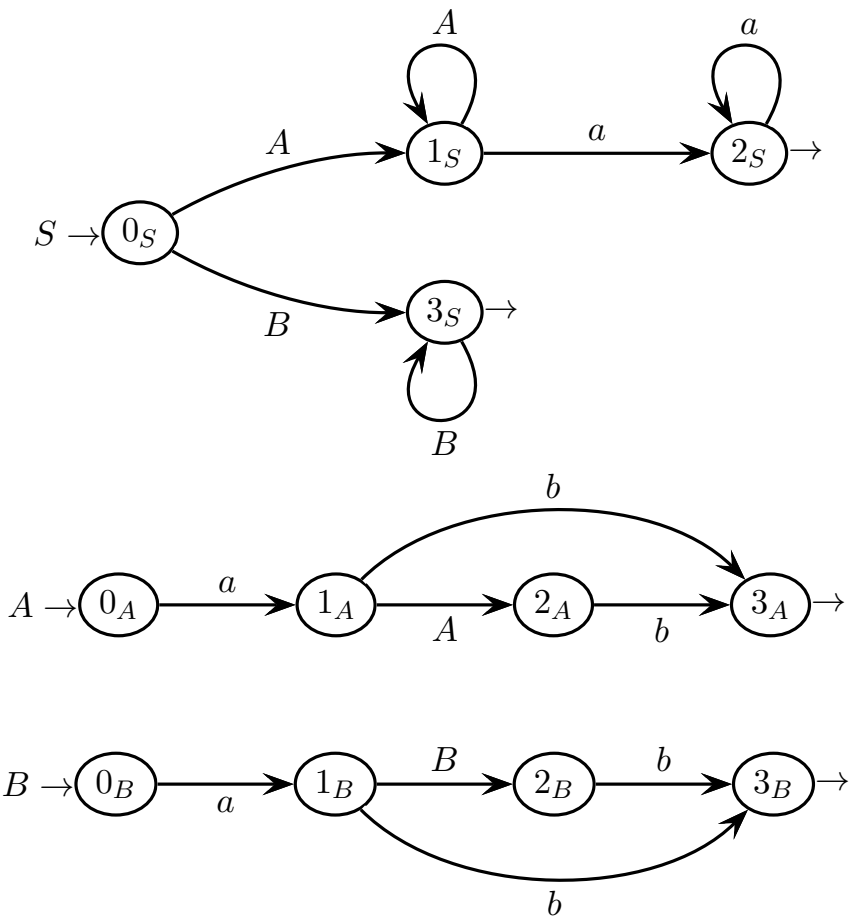
   ```
   B := A + b + c;              // parsed as A + (b + c)

   A := A + b + (c + d + B);    // parsed as A + b + ((c + d) + B)
   ```

   Write an extended (*EBNF*) grammar G', not ambiguous, that models this modified language fragment. You can reuse the grammar G obtained before, and just show the changes to be introduced.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G, represented as a machine net over the terminal alphabet $\Sigma = \{\ a,\ b\ \}$ and nonterminal alphabet $V = \{\ A,\ B,\ S\ \}$ (axiom S).



Answer the following questions:

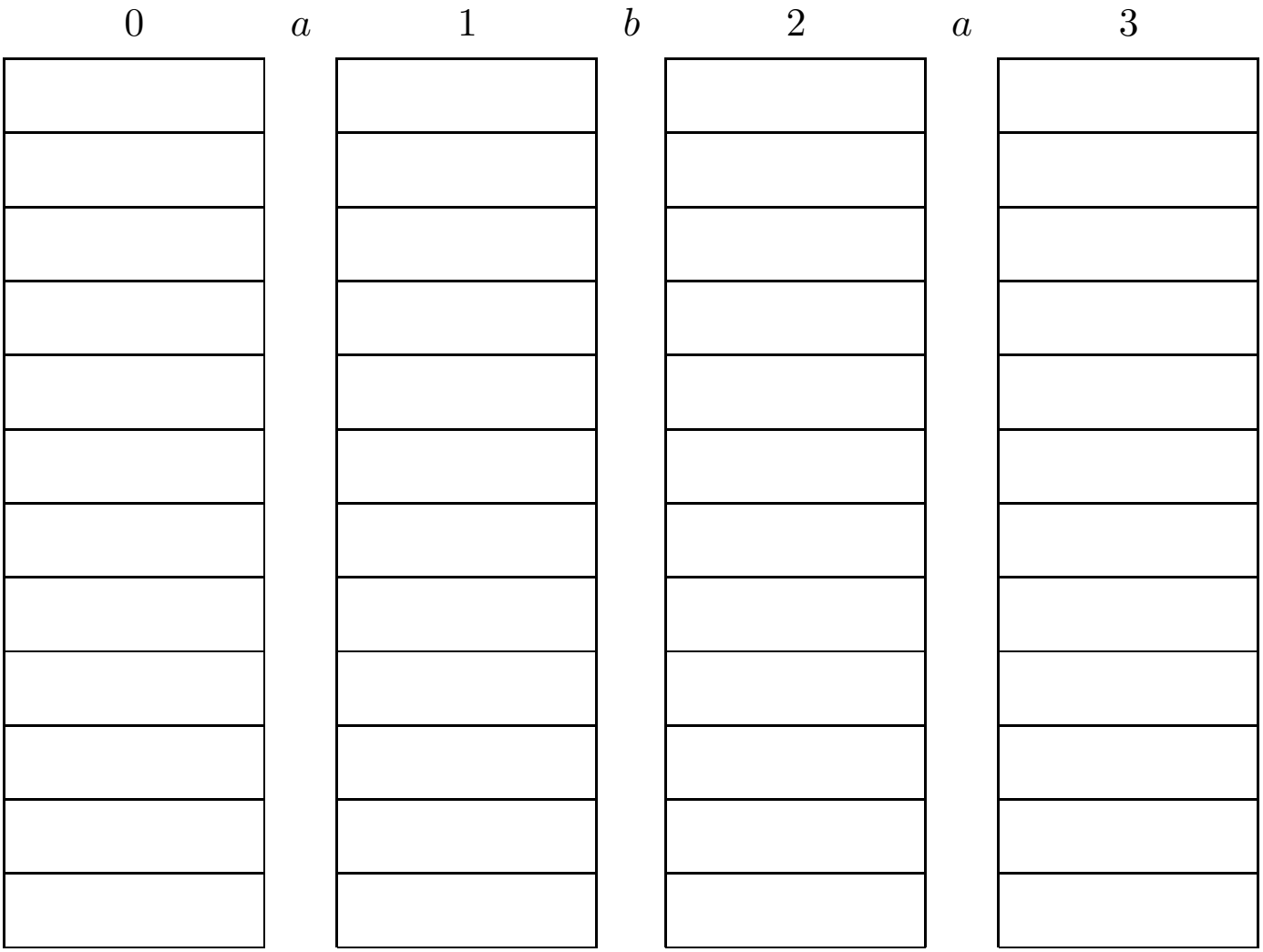
(a) Build a small part of the pilot automaton that suffices to prove that the language is not $ELL(1)$.

(b) Build a small part of the pilot automaton that suffices to prove that the language is not $ELR(1)$.

(c) By using the Earley method, simulate the analysis of the string $a\,b\,a$. Furthermore, on the data structure computed by the algorithm, show whether and why the string $a\,b\,a$, or any of its prefixes, is accepted. Use the Early table prepared on the next page.

(d) (optional) For language $L(G)$, define an $ELR(1)$ grammar G', as simple as possible, and show that this new grammar G' is actually $ELR(1)$.

Earley vector of string $a\ b\ a$ (to be filled)
(the number of rows is not significant)



4 Language Translation and Semantic Analysis 20%

1. It is well known that the following *EBNF* grammar G, not ambiguous (axiom S):

$$G\colon S \to \left(a\ S\ b \right)^*$$

generates the Dyck language with parentheses a (open) and b (closed).

Consider a natural generalization of the notion of translation grammar to the case of *Extended BNF* grammars, following the ideas underlying Regular Translation Expressions. Use the above grammar G to take inspiration for the following questions.

(a) Write a translation grammar (or scheme) G_1, not ambiguous, of *EBNF* type, that translates the source language $L(G)$ so that the parenthesis pairs at an even *depth* level (0, 2, 4, etc, notice that the depth index starts from 0) are transliterated onto c (open) and d (closed). The other parentheses are left unchanged by the translation. Samples:

$$\begin{aligned} \tau_1(a\,b\,a\,b) &= c\,d\,c\,d \\ \tau_1(a\ a\,b\,a\,b\ b) &= c\ a\,b\,a\,b\ d \\ \tau_1(a\ a\ a\,b\ b\,a\,b\ b) &= c\ a\ c\,d\ b\,a\,b\ d \end{aligned}$$

(b) Write a translation grammar (or scheme) G_2, not ambiguous, of *EBNF* type, that translates the source language $L(G)$ so that the parenthesis pairs at an even *position* (second, fourth, sixth, etc, notice that the position index starts from 1) in a list of parentheses concatenated at the same depth level are transliterated onto c (open) and d (closed). The other parentheses are left unchanged by the translation. Samples:

$$\begin{aligned} \tau_2(a\,b\,a\,b\,a\,b) &= a\,b\,c\,d\,a\,b \\ \tau_2(a\ a\,b\,a\,b\,a\,b\ b) &= a\ a\,b\,c\,d\,a\,b\ b \\ \tau_2(a\ a\,b\,a\,b\,a\,b\ b\,a\ a\,b\ b) &= a\ a\,b\,c\,d\,a\,b\ b\,c\ a\,b\ d \end{aligned}$$

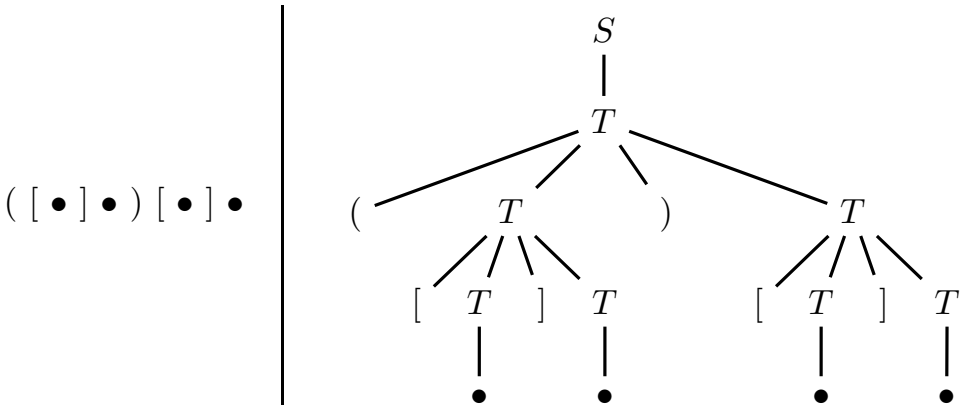
(c) (optional) Say if the translation grammar (or scheme) G_2 found at point (b) is deterministic or not, in particular if it is of type *ELL*, and justify your answer.

---

2. Consider the following abstract syntax, which generates well balanced parenthetical expressions (with two parenthesis types: round and square), where the bullets "$\bullet$" are enclosed in a number ≥ 0 of round or square parenthesis pairs (axiom S):

$$\begin{cases} 1: & S & \rightarrow & T \\ 2: & T & \rightarrow & (\ T\ )\ T \\ 3: & T & \rightarrow & [\ T\ ]\ T \\ 4: & T & \rightarrow & \bullet \end{cases}$$

For each bullet in the expression, let r and s be the numbers of enclosing *round* and *square* parenthesis pairs, respectively. Suppose each bullet is given a value v, such that $v = 2 \times r + 3 \times s$. The entire expression is also given a value, defined as the sum of the values of all its bullets. For instance, in the sample expression below:



the syntax tree of which is reported on the right, the value v of the expression is:

$$v = \underbrace{2+3}_{\text{first bullet}} + \underbrace{2+0}_{\text{second bullet}} + \underbrace{0+3}_{\text{third bullet}} + \underbrace{0+0}_{\text{fourth bullet}} = 10$$

Answer the following questions:

(a) Define an attribute grammar, based on the above syntax, that defines the computation of the value v for the entire expression, as explained above. It is suggested to use three attributes r, s, and v with the meaning previously explained. See the attribute table on the next page.

(b) Decorate the syntax tree of the sample expression $(\,[\,\bullet\,]\,\bullet\,)\,[\,\bullet\,]\,\bullet$ with the values of the attributes. Use the syntax tree prepared on the next page.

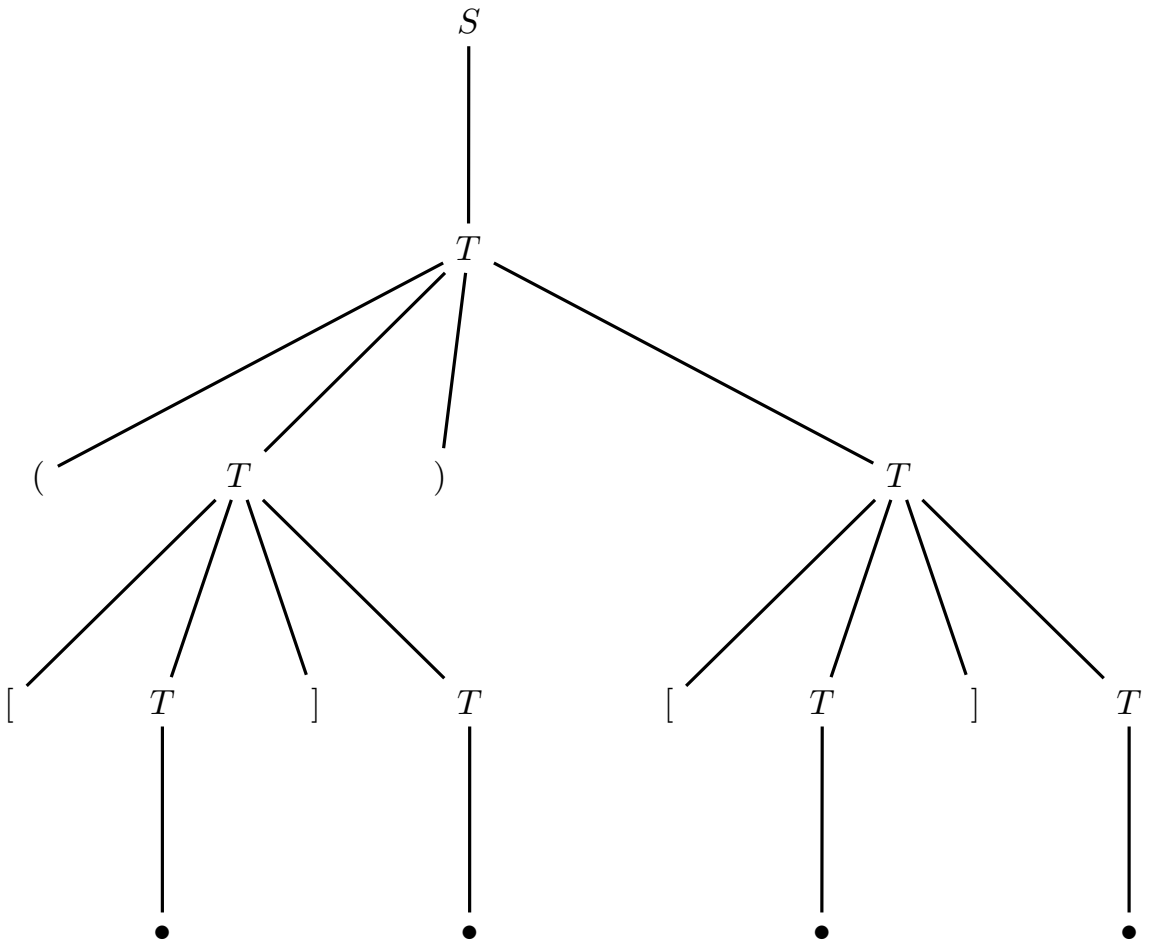
(c) (optional) Determine if the attribute grammar is of type one-sweep and if it satisfies the L condition, and reasonably justify your answers.

---

attributes assigned to be used

type	*name*	*domain*	*(non)term.*	*meaning*
right	r	integer	T	number ≥ 0 of pairs of round brackets that enclose the bullet or (sub)expression
right	s	integer	T	number ≥ 0 of pairs of square brackets that enclose the bullet or (sub)expression
left	v	integer	S, T	value ≥ 0 given to the bullet or (sub)expression

tree prepared for question (b)



#	*syntax*	*semantics*
1:	$S_0 \rightarrow T_1$	
2:	$T_0 \rightarrow (T_1) T_2$	
3:	$T_0 \rightarrow [T_1] T_2$	
4:	$T_0 \rightarrow \bullet$	